Innovative Evaluation Practices for BCSE308P - Computer Networks Lab

Fall Semester 2026-27

DESIGN AND IMPLEMENTATION OF A MULTI-THREADED HTTP/1.1 WEB SERVER FROM SCRATCH USING RAW TCP SOCKETS

24BCE0702 YASH PRADHAN

24BCE0714 ANJINI PANDEY

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



October 2026

ABSTRACT

Concurrent web servers must serve many TCP clients with a bounded number of threads and a bounded queue, and when offered load exceeds capacity, queueing delay dominates tail latency. This project implements, from scratch in C17 on raw POSIX TCP sockets and pthreads, a multi-threaded HTTP/1.1 web server consisting of a listener, a bounded connection queue, a fixed pool of four workers, a streaming parser for a documented subset of RFC 9112, a router, sendfile-based static delivery, a metrics endpoint and a live dashboard. On top of it, Queue-and-Latency-Aware Admission Control (QLAAC) was implemented: a three-state controller (NORMAL, CAUTION, OVERLOAD) that samples worker utilisation, queue occupancy and recent p95 latency every second, shortens keep-alive reuse in CAUTION, rejects new connections with HTTP 503 in OVERLOAD, and recovers with hysteresis. The baseline is the same binary with the controller disabled. A strict benchmark harness separated 2xx responses, 5xx responses, timeouts and transport errors. Five 10-second trials were run at 2, 8, 32 and 64 closed-loop clients, and results are reported as mean ± sample standard deviation. At 2 clients QLAAC showed no significant difference. At 8, 32 and 64 clients it reduced the p99 latency of successful responses by 87.8%, 91.9% and 63.4%, but successful throughput fell by 67.9%, 71.3% and 90.7%, median latency rose, and at 64 clients 72.1% of completed responses were 503 rejections and p99 varied widely between trials (161.2 ± 159.9 ms). QLAAC therefore trades throughput for tail latency, and its p95 signal excluded queueing delay in this workload. All 49 automated tests (26 C, 23 Python) pass with a server running. Results come from a single-host loopback testbed with a Python load generator and should not be generalised to production.

Certificate

This is to certify that the project report titled "Design and Implementation of a Multi-Threaded HTTP/1.1 Web Server from Scratch Using Raw TCP Sockets" is a bonafide record of the work done by **YASH PRADHAN (24BCE0702)** and **ANJINI PANDEY (24BCE0714)** in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

Name and Signature of the Students

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	2
1.	**INTRODUCTION**	6
	1.1 Background and motivation	6
	1.2 Problem statement	6
	1.3 Objectives	7
	1.4 Scope and limitations	7
	1.5 Mapping to course modules and the lab experiment	8
2.	**Literature Survey**	9
	2.1 Overview of the existing protocol or scheme	9
	2.2 Review of related work	9
	2.3 Research gap and justification of novelty	10
3.	**Proposed System**	12
	3.1 System architecture or topology	12
	3.2 Baseline mechanism description	12
	3.3 Proposed mechanism: algorithm, pseudocode and flowchart	13
	3.4 Novelty statement	16
	3.5 Assumptions and design decisions	16
4.	**Implementation**	17
	4.1 Tools, platform, versions and hardware	17
	4.2 Simulation or testbed setup	17
	4.3 Code structure and explanation of key modules	18
	4.4 Challenges faced and how they were solved	20
5.	**Experimental Setup and Performance Evaluation**	22
	5.1 Performance metrics and why they were chosen	22
	5.2 Scenarios and parameter values	22
	5.3 Results: tables and graphs, baseline vs proposed	23
	5.4 Analysis and discussion	26
	5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails	28

6. Conclusion and Future Work 30

6.1 Summary of work and key findings 30

6.2 Limitations 30

6.3 Future enhancements 31

7. References 32

References (IEEE style) 32

Appendix A: Important source code or configuration listings 33

Appendix B: Additional screenshots, packet captures, raw result tables 35

1. INTRODUCTION

1.1 Background and motivation

The World Wide Web is built on a simple exchange: a client opens a TCP connection to a server, sends an HTTP request and receives a response. HTTP/1.1 is specified in RFC 9112 for message syntax and framing and in RFC 9110 for semantics [1], [2], while TCP provides the reliable, ordered byte stream on which the messages travel [3]. TCP delivers bytes, not messages: a single read on a socket can return part of a header, exactly one request, or several pipelined requests, so every HTTP server must buffer the stream and find message boundaries itself.

A server rarely serves one client at a time. Each accepted connection needs a thread of control for as long as the server is reading a request, producing a response or waiting on an idle persistent connection. Because threads, memory and file descriptors are finite, practical servers bound the resources they spend: a fixed-size pool of worker threads and a bounded queue of accepted-but-unserved connections. This bounded design protects the host, but it creates a queue. By Little's law, the mean number of requests in a stable system equals the arrival rate multiplied by the mean time in the system [11]. For a closed population of C clients, the mean response time therefore grows in proportion to C divided by the throughput the server can sustain, so once all workers are busy additional clients add waiting time rather than throughput. Long queues in front of a saturated resource are the cause of "bufferbloat" in network devices [9], and the resulting tail of slow requests matters disproportionately to users of large systems [10].

This project explores that relationship at the scale of a single machine. It builds a complete multi-threaded HTTP/1.1 server directly on raw TCP sockets, with no third-party HTTP or event library, so that every layer between the socket and the response is visible and measurable. It then adds an experimental overload controller, Queue-and-Latency-Aware Admission Control (QLAAC), and evaluates it against the same server running a fixed policy.

1.2 Problem statement

A thread-pool web server with a fixed number of workers W and a bounded connection queue of capacity Q has two failure modes under load. If clients hold persistent connections, each worker is occupied by one connection at a time, and connections beyond W wait in the queue. Requests that arrive on a waiting connection see a delay equal to the service time of the

connections ahead of them, so the tail of the latency distribution grows even though the median remains small. If the queue fills, new connections can only be refused. A fixed policy neither anticipates this nor distinguishes a harmless busy period from a developing overload.

The engineering problem addressed here is therefore twofold: (i) to implement a correct, bounded, concurrent HTTP/1.1 server on raw sockets; and (ii) to determine, with trustworthy measurements, whether a lightweight feedback controller that uses signals already available inside the server (worker utilisation, queue occupancy and recent latency) can protect tail latency under overload, and at what cost in successful throughput and rejected requests.

1.3 Objectives

The following objectives were pursued and are implemented in the delivered code:

- Implement a TCP listener using the POSIX socket API (socket, bind, listen, accept, poll) with explicit handling of partial reads and writes.
- Implement a thread-safe bounded FIFO connection queue (mutex and condition variables) and a fixed pool of pthread workers that own each accepted connection.
- Implement a streaming parser for a documented subset of HTTP/1.1 (RFC 9112), with Content-Length framing, persistent connections, pipelined-byte preservation and explicit error statuses for unsupported features.
- Implement routing for a health endpoint, a JSON metrics endpoint, an echo endpoint and secure static-file delivery using sendfile(2) with path-traversal checks.
- Implement runtime telemetry (counters and a rolling latency reservoir) and a browser dashboard that reads it.
- Design and implement QLAAC, a three-state admission controller with hysteresis, and compare it against a fixed-policy baseline (same binary, controller disabled).
- Build a benchmark harness whose accounting separates successful 2xx responses from other outcomes, and evaluate both configurations over repeated trials at several concurrency levels.
- Verify the implementation with an automated test suite and report limitations honestly.

1.4 Scope and limitations

Delivered: a Linux/POSIX server in C17 (about 3,160 lines in src/ and 1,070 lines in include/ by wc -l), GET, HEAD and POST handling within the limits described in Chapter 4, Content-Length framed request bodies up to 10 MiB, persistent connections with a configurable request

limit and idle timeout, bounded queue and worker pool, QLAAC, metrics, dashboard, 49 automated tests, and a Python benchmark harness.

Outside the scope or not implemented: Transfer-Encoding: chunked (answered with 501), Expect: 100-continue (answered with 417), TLS/HTTPS, HTTP/2 and HTTP/3, compression, caching validators and range requests, virtual hosts, authentication, and event-driven I/O (epoll). Full RFC 9112 compliance, production-grade security and scalability beyond the tested range are **not** claimed: the evidence in this report supports only the behaviours named here. The performance evaluation is limited to one workload (GET /health), one host and four concurrency levels (Section 5.2).

1.5 Mapping to course modules and the lab experiment

Because an official BCSE308P syllabus or lab-experiment specification document was not archived within the project repository, specific curriculum module numbers and official laboratory experiment titles could not be independently verified from repository files and are not fabricated here. Instead, Table 1.1 formally maps each implemented system component to foundational Computer Networks concepts (including transport-layer socket abstractions, application-layer HTTP framing, thread-pool concurrency models, queueing dynamics, and empirical performance evaluation) that the implementation concretely exercises.

Table 1.1: Mapping of project components to Computer Networks concepts

Computer Networks concept	Where it appears in the project
Transport layer: TCP connection lifecycle and byte-stream semantics [3]	src/tcp_socket.c: listen backlog, accept, partial recv/send loops with poll deadlines, shutdown(SHUT_WR) and lingering close
Application layer: HTTP/1.1 message syntax, framing and persistent connections [1], [2]	src/http_parser.c, src/http_response.c: request line and header parsing, Content-Length framing, keep-alive, 400/405/417/501/505-class errors
Client–server model and concurrency	src/thread_pool.c, src/connection_queue.c: acceptor thread, bounded FIFO, fixed worker pool
Queueing delay, buffering and overload (cf. [9], [11])	Bounded queue (capacity 64), QLAAC thresholds, 503 with Retry-After [2]
Network performance measurement	benchmark/run_experiment.py: throughput, latency percentiles, status-class accounting
Network application security	src/static_files.c: traversal rejection and canonical-root check; parser size limits; tests/test_security.py

2. LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

HTTP/1.1. RFC 9112 defines the HTTP/1.1 message format: a request line, header fields, an optional body delimited by Content-Length or chunked transfer coding, and rules for persistent connections, in which several request/response exchanges reuse one TCP connection unless either side sends "Connection: close" [1]. RFC 9110 defines method and status semantics, including the 503 (Service Unavailable) status, which signals temporary overload or maintenance, and the Retry-After header field that a server may send with it [2]. Neither document prescribes how a server should decide when to refuse work; that policy is left to implementations.

TCP. RFC 9293 consolidates the TCP specification: a connection-oriented, reliable, ordered byte stream with flow control [3]. A server calls listen with a backlog, and the kernel completes the three-way handshake for connections that sit in the accept queue until the application calls accept. This is why, in the experiments, a client can observe a connection as established while the server application has not yet picked it up.

Server architectures and overload handling. Thread-per-connection and thread-pool servers are simple but block a thread per active connection. Event-driven designs multiplex many connections on few threads, and hybrids such as staged event-driven architecture (SEDA) split request processing into stages joined by bounded queues, with per-stage controllers that adapt resource use and perform admission control [4], [5]. Comparative studies show that careful tuning matters as much as architecture [7]. Under overload, scheduling approaches such as shortest-remaining-processing-time ordering of connections can improve response times without refusing work [6], whereas admission-control approaches deliberately refuse some work in order to protect the rest [5]. In networking, active queue management such as CoDel uses queueing delay rather than queue length as the control signal, because long standing queues, not bursts, are harmful [8], [9].

2.2 Review of related work

Table 2.1 summarises the standards and prior work most relevant to this project. The "Gaps" column states what each item does not provide with respect to the present problem, not a criticism of its own goals.

Table 2.1: Literature survey

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	Fielding, Nottingham, Reschke (Eds.), 2022 [1], [2]	HTTP/1.1 message syntax and framing; HTTP semantics, 503 and Retry-After	Normative specification (no experimental metrics)	Authoritative definition of framing, persistent connections and overload status codes	Does not prescribe an overload or admission policy for servers
2	Welsh, Culler and Brewer, 2001 [4]	SEDA: stages connected by bounded queues with dynamic resource controllers	Throughput and response time of a SEDA-based web server compared with conventional servers	Explicit queues make load visible; per-stage control	Requires a staged event architecture; far more complex than a flat thread pool
3	Welsh and Culler, 2003 [5]	Adaptive overload control: admission control driven by measured response time of each stage	Response-time percentile against a target; SEDA-based services	Feedback on measured latency; request-level load shedding	Depends on the SEDA framework and stage instrumentation
4	Schroeder and Harchol-Balter, 2006 [6]	Size-based (SRPT-style) connection scheduling for static content under overload	Mean response time under overload; web-server testbed	Improves response time without refusing requests	Needs known response sizes; does not bound queue or protect against saturation
5	Pariag et al., 2007 [7]	Comparison of thread-per-connection, event-driven and hybrid server architectures	Throughput and response time under load; tuned server implementations	Shows tuning, not architecture alone, determines performance	Does not study admission control or overload shedding
6	Nichols and Jacobson, 2012 [8]; Gettys and Nichols, 2011 [9]	Delay-based active queue management (CoDel); analysis of excessive buffering	Queueing delay (sojourn time) in packet buffers; simulation and real traffic	Control on delay rather than length; motivates measuring delay	Targets packet queues in routers, not application connection queues

2.3 Research gap and justification of novelty

Baseline limitation. A thread-pool server with a fixed bounded queue and a fixed keep-alive policy treats every load level identically. In the measurements of this project, that baseline sustained its throughput as the number of clients grew from 8 to 64, but the p99 latency of successful responses grew from 29.1 ms to 440.3 ms, because connections waited behind pinned workers (Section 5.4). The queue never overflowed in the baseline trials, so the only protection it offers (a 503 when the queue is full) never activated. Established admission-control frameworks [4], [5] address such behaviour but are built around staged architectures and are heavy for a teaching-scale server.

What this project adds. The contribution is an implemented, instrumented and honestly evaluated composition of established ideas in a from-scratch C server: (i) a transparent three-state controller whose inputs (worker utilisation, queue occupancy and recent p95 latency) are all available inside a flat thread-pool server; (ii) two levels of actuation: shortened keep-alive reuse for existing connections, and accept-time 503 rejection with Retry-After for new connections; (iii) hysteresis on recovery; and (iv) a benchmark harness whose accounting separates 2xx responses, 503 shedding, timeouts and transport errors. The novelty claim is

deliberately limited. Latency- and queue-triggered admission control is not new [5], and delay-based queue signals come from [8]. The project does not claim a new algorithm class and, as Chapter 5 shows, it does not claim that QLAAC is better in every respect; its value lies in making the throughput/tail-latency/rejection trade-off of a concrete implementation measurable and explainable.

3. PROPOSED SYSTEM

3.1 System architecture or topology

Figure 3.1 shows the request path. A single listener thread polls the listening socket (100 ms poll timeout so it can notice shutdown), accepts connections and consults the QLAAC state. In OVERLOAD it writes a fixed 503 response and closes the connection itself. Otherwise it pushes the connection onto a bounded FIFO queue without blocking; if the queue is full it writes a 503 with Retry-After: 2. Four pre-spawned worker threads pop connections, and each worker keeps its connection until it ends. Within that connection the worker repeatedly parses a request with the streaming parser, dispatches it through the router (health, metrics, echo or static file) and sends the response, then records latency and status in the metrics collector. A separate sampler thread wakes every 1000 ms, reads queue occupancy, worker utilisation and the rolling p95 latency, and updates the QLAAC state.

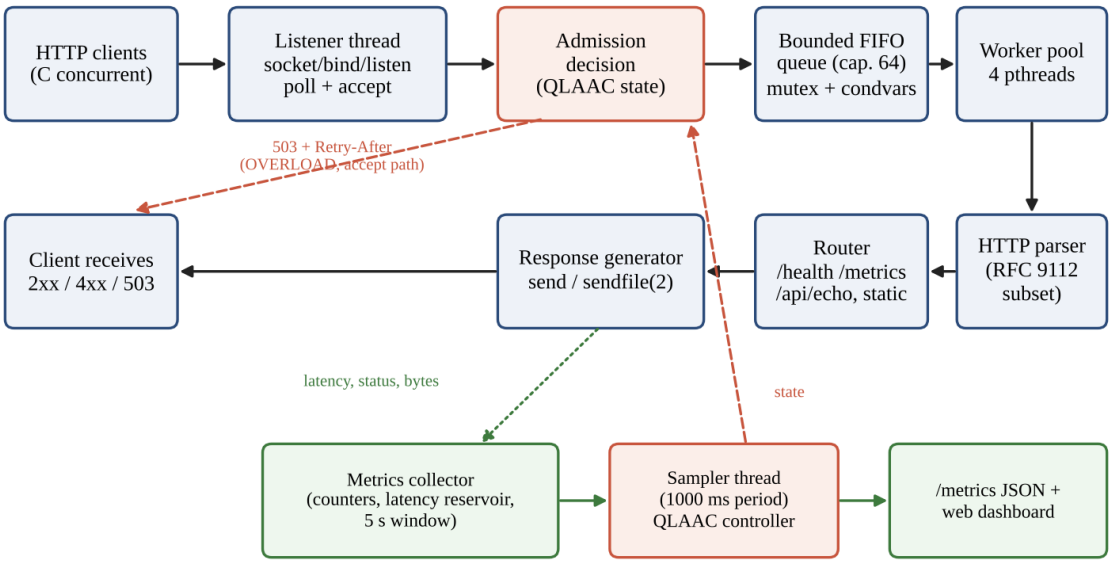


Figure 3.1: Request-handling architecture: client, listener, admission decision, bounded queue, workers, parser, router, response generator, and the metrics/controller feedback path (drawn from src/server.c, thread_pool.c and the architecture description).

3.2 Baseline mechanism description

The baseline is **the same executable with adaptive_control_enabled = false** (the default). The controller update is never called, adaptive_controller_evaluate always returns ADMIT, and no keep-alive throttling occurs. The baseline therefore uses a fixed policy:

- 4 worker threads (server_config_init_defaults); connection queue capacity 64; listen backlog 128.
- Persistent connections allowed for up to 100 requests per connection and 5 s idle timeout.
- If the bounded queue is full, the listener replies 503 (Retry-After: 2) and closes. This path is part of the baseline; it never triggered in the baseline trials (5xx count 0 in all 20 trials).
- No latency feedback, no early shedding, no keep-alive adaptation.

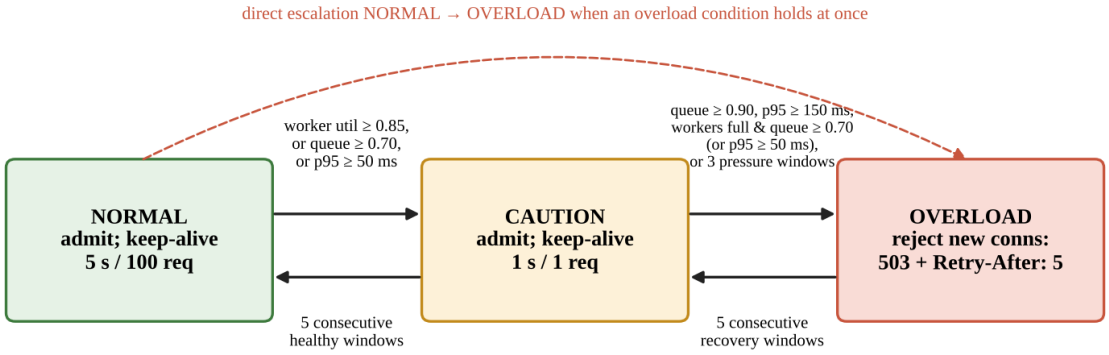
The baseline is **not** an unbounded queue. An earlier summary in the project package described it that way, but src/connection_queue.c enforces the capacity limit and the harness never exceeded it: with at most 64 clients, at most 60 connections can wait while four are in service.

3.3 Proposed mechanism: algorithm, pseudocode and flowchart

QLAAC runs in the sampler thread once per sampling window (1000 ms). It receives the queue occupancy ratio q (current depth divided by capacity 64), the worker utilisation u (active workers divided by 4) and the p95 latency of recent samples. The p95 is computed from a 1,024-slot reservoir restricted to the last 5 seconds of completed requests; the latency of each request is measured in the worker from the moment request bytes are available to the completion of the send. The controller derives six flags from the thresholds in Table 3.1 and then transitions between the states shown in Figure 3.2.

Table 3.1: QLAAC signals and thresholds (defaults in server_config_init_defaults and config/server.conf.example)

Signal / parameter	Threshold	Meaning
Worker utilisation u	caution ≥ 0.85; saturated ≥ 1.00	Active workers divided by total workers
Queue occupancy q	growing ≥ 0.70; critical ≥ 0.90	Queue depth divided by capacity (64)
Recent p95 latency	elevated ≥ 50 ms; critical ≥ 150 ms	p95 of requests completed in the last 5 s
Sustained-pressure windows	3	Consecutive windows of moderate stress before OVERLOAD
Recovery windows	5	Consecutive healthy windows to step down one state
Sampling period	1000 ms	Controller evaluation interval



Evaluated once per sampling window (1000 ms) by the sampler thread; thresholds from server.conf.example.

Figure 3.2: QLAAC state machine (from src/adaptive_controller.c). CAUTION shortens keep-alive reuse; OVERLOAD rejects new connections at accept time.

Decision rules (adaptive_controller_update_ex). Define worker_saturated = (u ≥ 1.0), worker_caution = (u ≥ 0.85), queue_growing = (q ≥ 0.70), queue_critical = (q ≥ 0.90), latency_elevated = (p95 ≥ 50 ms) and latency_critical = (p95 ≥ 150 ms). A window counts toward sustained pressure when (saturated and q ≥ 0.5) or (saturated and latency_elevated) or (queue_growing and latency_elevated). The overload condition holds if queue_critical, or latency_critical, or (saturated and queue_growing), or (saturated and latency_elevated), or the sustained-pressure counter has reached 3. The caution condition holds if the overload condition holds or any of queue_growing, latency_elevated, worker_caution or worker_saturated is true. Worker saturation alone, with an almost empty queue and low latency, therefore produces CAUTION (an early warning) but not OVERLOAD.

Actuation. In CAUTION each worker, before reading the next request on a connection, sets its keep-alive timeout to 1 s and its request limit to 1, so the connection is closed after the next response and the worker returns to the queue. In OVERLOAD the listener answers every newly accepted connection with "503 Service Unavailable", "Retry-After: 5" and "Connection: close", without queuing it. Connections already held by workers are not interrupted in OVERLOAD (the worker loop applies only the CAUTION throttle).

Recovery. From OVERLOAD, five consecutive windows in which the overload condition does not hold are required to step down to CAUTION; from CAUTION, five consecutive windows with no flag raised are required to return to NORMAL. Escalation is immediate (a single window), de-escalation is slow, which prevents oscillation but also prolongs rejection.

Pseudocode (one sampling window):

```
function QLAAC_UPDATE(q, u, p95):
    sat      <- (u >= 1.00);    wcaut <- (u >= 0.85)
    qgrow    <- (q >= 0.70);    qcrit <- (q >= 0.90)
    lelev    <- (p95 >= 50);    lcrit <- (p95 >= 150)       # p95 ignored if 0
    cand <- (sat and q >= 0.5) or (sat and lelev) or (qgrow and lelev)
    pressure <- cand ? pressure + 1 : 0
    over  <- qcrit or lcrit or (sat and qgrow) or (sat and lelev) or pressure >= 3
    caut  <- over or qgrow or lelev or wcaut or sat
    if over:
        state <- OVERLOAD ; healthy <- 0
    elif caut:
        if state == OVERLOAD: healthy++ ; if healthy >= 5: state <- CAUTION; healthy <- 0
        elif state == NORMAL: state <- CAUTION ; healthy <- 0
        else: healthy <- 0
    else:
        if state == OVERLOAD: healthy++ ; if healthy >= 5: state <- CAUTION; healthy <- 0
        elif state == CAUTION: healthy++ ; if healthy >= 5: state <- NORMAL; healthy <- 0
        else: state <- NORMAL
```

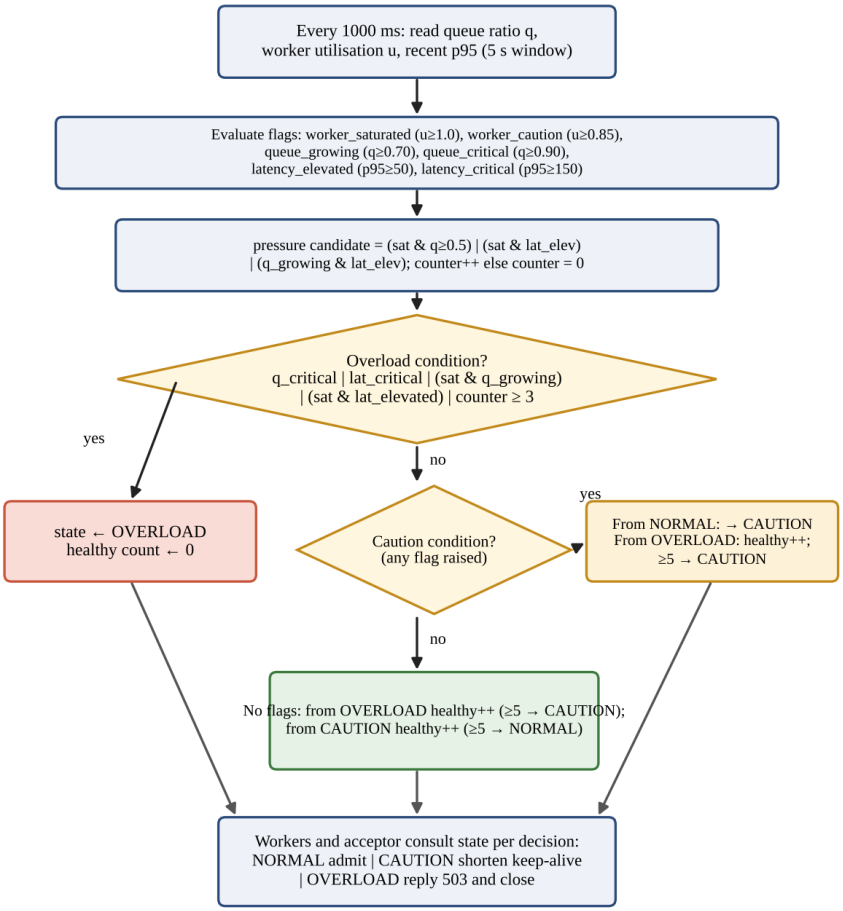


Figure 3.3: Flowchart of one QLAAC evaluation window (condensed from adaptive_controller_update_ex).

3.4 Novelty statement

QLAAC differs from the baseline in exactly four implemented ways: (1) a feedback loop (sampler thread plus controller) that the baseline does not run; (2) keep-alive shortening for connections in service while the state is CAUTION, whereas the baseline always allows up to 100 requests and 5 s idle per connection; (3) accept-time 503 rejection in OVERLOAD, before the connection occupies a queue slot, whereas the baseline rejects only when the queue is completely full; and (4) hysteresis on recovery. Everything else (worker count, queue capacity, parser, router, response generation) is identical because it is the same binary. The controller is a composition of established ideas (Section 2.3) and is evaluated as an engineering mechanism, not presented as a new class of algorithm.

3.5 Assumptions and design decisions

Table 3.2: Design decisions and assumptions (values verified in the source and configuration files)

Decision	Value	Rationale / assumption
Worker threads	4	Default in server.c; fixed at start. Assumed small to make saturation reachable on a laptop-class host; not tuned (sizing study not performed).
Queue capacity	64 connections	Bounds memory; equals the highest client count tested, so the baseline queue cannot overflow in closed-loop tests with $C \leq 64$.
Listen backlog	128	Kernel accept queue sized above the application queue.
Keep-alive (normal)	100 requests, 5 s idle	Defaults; allows connection reuse. The harness itself closes a connection after 50 requests.
Keep-alive (CAUTION)	1 request, 1 s	Frees workers quickly so that waiting connections get a turn.
Sampling window	1000 ms	Coarse relative to request times (about 0.1 ms); a window can contain thousands of requests.
Latency window / reservoir	5 s / 1,024 samples	metrics.h constants; the p95 reflects only recent requests.
Overload response	503 + Retry-After: 5 (queue-full: 2)	Semantics of RFC 9110 [2].
Measurement start	Request bytes available (not connection accept)	Hence queue wait before a worker picks up the connection is not part of the server-side latency signal (Section 5.5).

Two configuration details are worth recording. First, config/server.conf.example sets max_header_size and io_buffer_size to 8,192 bytes, while the compiled server defaults (server_config_init_defaults in src/server.c) initialize both to 16,384 bytes; when the server is started without an explicit configuration file path, the compiled 16,384-byte buffers are active. Second, the key worker_idle_timeout_sec in config/server.conf.example is not parsed by server_config_load_file and therefore has no effect.

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Table 4.1: Implementation environment

Item	Value	Source / status
Language and standard	C17 (-std=c17), POSIX.1-2008 (-D_POSIX_C_SOURCE=200809L), pthreads	Makefile
Compiler warnings	-Wall -Wextra -Wpedantic -Wformat=2 -Wshadow and others	Makefile
OS-specific features	Linux sendfile(2) for static files	src/tcp_socket.c
Libraries	libc and POSIX only; no third-party HTTP or event library	Source inspection
Benchmark harness	Python 3 standard library only (socket, threading); closed-loop clients	benchmark/run_experiment.py
Student development/benchmark platform	Test log states "Linux (POSIX C17 / GCC 11+ via WSL2 / Windows 11 Host)"; screenshot notes mention a WSL2 Debian environment	results/test_evidence/test_run_summary.txt; exact compiler version not recorded
Benchmark host hardware specifications	Not recorded in archived benchmark CSV headers; Windows 11 host with WSL2 execution environment	Archived benchmark CSV and testbed inspection
Independent verification environment (tests & verification)	Intel(R) Core(TM) Ultra 7 155H (16 physical cores, 22 vCPUs), 7.5 GiB RAM, Linux 6.18.33.2-microsoft-standard-WSL2 (x86_64), GCC 15.2.0, Python 3.14.4 (October 2026)	Direct system verification via wsl lscpu / uname / gcc / python3 (Table 4.1; Appendix B.2)

The benchmark results in Chapter 5 were produced on the original development platform (Windows 11 host running Linux under WSL2 with GCC 11+), where specific CPU model and memory limits were not logged into the CSV headers. The independent verification environment detailed in Table 4.1 (Intel Core Ultra 7 155H, Linux 6.18 WSL2 kernel, GCC 15.2.0, Python 3.14.4) was used to re-execute the test suite (Section 6 / Appendix B.2), validate the statistical calculations, and perform a qualitative mechanism check (Appendix B.6) whose numbers are not mixed into Chapter 5.

4.2 Simulation or testbed setup

The testbed is a single host: the server listens on 127.0.0.1:8080 and the Python load generator runs on the same machine, so no physical network or wire latency is involved. No network emulation was used. Each experiment configuration is one server process. The harness opens C threads; each thread repeatedly opens a TCP connection, sends up to 50 sequential GET requests with "Connection: keep-alive", records the client-observed latency of each response

(from just before sendall to the last body byte), closes the connection and opens a new one until the trial duration elapses. If a response carries "Connection: close" the thread reconnects immediately. The workload is closed-loop: a client issues its next request only after the previous response, and after a 503 it reconnects at once with no back-off.

Table 4.2: Testbed and experiment parameters (from benchmark/run_experiment.py and the raw CSV files)

Parameter	Value
Target endpoint	GET /health (small JSON body of 89 bytes)
Concurrency levels (clients C)	2, 8, 32, 64
Trials per configuration	5 independent 10-second trials per (configuration, C); 20 trials per configuration
Configurations	Baseline: QLAAC disabled; Proposed: QLAAC enabled (server flag -a or adaptive_control_enabled = true)
Server settings	4 workers, queue 64, backlog 128, keep-alive 100 requests / 5 s (defaults); QLAAC thresholds as in Table 3.1
Client behaviour	Up to 50 requests per connection; 2 s connect/read timeout; immediate reconnect; 0.5 s pause between trials
Data files	results/raw/experiment_baseline_tuned_comparison.csv and experiment_qlaac_tuned_comparison.csv
Benchmark execution command	python3 benchmark/run_experiment.py --url http://127.0.0.1:8080/health --clients 2,8,32,64 --duration 10 --trials 5 --output <csv> (with server bin/server [-a])

The final benchmark runs were executed using the v2.0 benchmark harness (benchmark/run_experiment.py) against bin/server (baseline) and bin/server -a (QLAAC). In the raw dataset filenames, 'tuned' denotes that the controller evaluated was the refined QLAAC implementation in src/adaptive_controller.c—where worker saturation alone acts as CAUTION while OVERLOAD requires elevated queue occupancy (q >= 0.90), high latency (p95 >= 150 ms), or sustained multi-factor pressure—superseding the preliminary controller that tripped overload on transient worker saturation alone. Because run_experiment.py steps through client counts sequentially with a 0.5 s inter-trial pause, controller state and hysteresis can carry over across consecutive client levels if the server process was not restarted externally between batches (Section 5.5).

4.3 Code structure and explanation of key modules

Table 4.3: Source modules (line counts from wc -l)

File	Lines	Responsibility
src/main.c	145	Command-line flags (-c, -p, -t, -d, -a), configuration loading, signal handling (SIGINT/SIGTERM), startup banner

File	Lines	Responsibility
src/server.c	388	Defaults and config-file parser; accept loop with poll; QLAAC admission at accept; queue push; 503 shedding; sampler thread
src/tcp_socket.c	478	Listener creation (getaddrinfo, SO_REUSEADDR), non-blocking accept, poll-based recv/send with deadlines, sendfile loop, lingering close
src/connection_queue.c	190	Thread-safe bounded circular FIFO; non-blocking push; timed pop; shutdown broadcast
src/thread_pool.c	257	Fixed pthread pool; per-connection keep-alive loop; request timing; CAUTION throttle; worker activity count
src/http_parser.c	435	Streaming request parser: request line, headers (max 64), URI ≤ 2,048 B, Content-Length framing, Host check, conflict and unsupported-feature errors
src/http_response.c	331	Status phrases, standard headers (Date, Server, Connection), error pages, body/file response sending
src/router.c	175	Dispatch: /health, /metrics (JSON), /api/echo (POST), static files
src/static_files.c	180	MIME types, ".." and %2e checks, realpath canonical-root check, directory-listing refusal, open and sendfile
src/adaptive_controller.c	211	QLAAC state machine, hysteresis, admission decision, shed counter
src/metrics.c	249	Atomic-style counters under lock, 1,024-sample latency reservoir over a 5 s window, p50/p95/p99 snapshot, JSON export
src/util.c	124	Monotonic time, RFC 1123 date formatting, logging
public/index.html	1,449	Single-page dashboard (HTML/CSS/JavaScript) polling /metrics
tests/ (C: 5 files, Python: 4 files)	771 + 501	Unit tests for controller, parser, queue, responses, metrics; protocol and benchmark-accounting tests

Listener and TCP handling. The listener uses a non-blocking socket and poll. tcp_recv_some and tcp_send_all loop until the requested number of bytes has moved or a monotonic deadline passes, which handles short reads and writes explicitly. Static files are sent with a sendfile loop. After a response is complete the worker calls shutdown(SHUT_WR) and waits briefly for the peer before closing, so that unread client data does not cause a reset.

Queue and thread pool. The queue is a circular buffer guarded by one mutex and two condition variables. The listener pushes with a zero timeout, so a full queue is detected immediately and the connection is rejected instead of blocking the accept loop. Workers pop with a one-second timeout so they can observe shutdown. A worker owns its socket exclusively for the lifetime of the connection and increments an active-worker count used by the controller and the dashboard.

Parser, routing and static files. The parser accumulates bytes in a per-connection buffer, returns "need more data" until headers (and the Content-Length body) are complete, and leaves any extra bytes in the buffer for the next pipelined request. It requires a Host header, rejects conflicting Content-Length and Transfer-Encoding, answers chunked bodies with 501 and Expect: 100-continue with 417, rejects bare line feeds and oversize headers, and treats HTTP/1.0 as non-persistent by default. The router serves GET/HEAD on /health and /metrics, POST on /api/echo, and static files from the public root; static paths containing ".." or encoded

dot-dot sequences are refused with 403 and the canonical path must remain inside the canonical root.

Metrics and dashboard. Each request records status class, latency, bytes received and bytes sent. 503 responses written by the listener increment separate overload counters and are deliberately not counted as completed requests. The /metrics route (excluded from latency statistics) returns counters, queue depth, active workers, rolling throughput, percentiles and the QLAAC state as JSON. Figure 4.1 shows the dashboard reading these values.

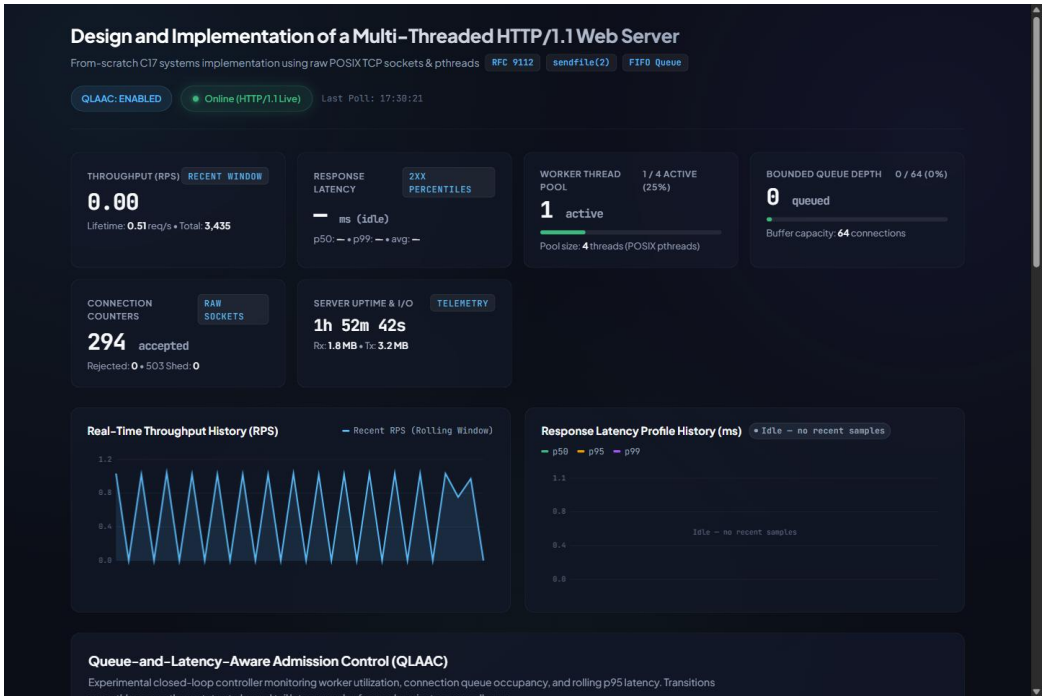


Figure 4.1: Live dashboard at idle (QLAAC enabled, 4 workers, queue 0/64, no recent latency samples), captured from the running server. Further captures are in Appendix B.1.

4.4 Challenges faced and how they were solved

The challenges below are documented in the repository or demonstrable from its code, tests and data files. No development history beyond that is claimed.

Table 4.4: Challenges, solutions and evidence

Challenge	Solution	Evidence
TCP gives a byte stream; one read may contain a partial or several requests	Per-connection buffer, streaming parser with a "need more data" state and preservation of pipelined leftover bytes	test_parser.c: test_pipelined_leftover_preservation; benchmark harness uses the same leftover-buffer rule
Early benchmark scripts counted any response beginning "HTTP/1.1" as success, so 503 rejections inflated throughput	Harness rewritten to parse the status line, enforce Content-Length framing and count 2xx, 3xx, 4xx, 5xx, timeouts, transport errors and invalid responses separately; throughput uses only 2xx	docs/experiment-methodology.md section 7; legacy CSVs kept but excluded (Appendix B.5); test_benchmark_accounting.py (11 tests)
Percentiles are undefined when a trial has no 2xx response	Reported as N/A instead of 0.00 ms	test_zero_success_reports_na_latency; N/A handling in run_experiment.py

Challenge	Solution	Evidence
A 503 written by the listener is not a processed request	Separate overload counters; "completed" is not incremented; a failed 503 write is not counted as sent	test_server_metrics.c: overload accounting and failed-write tests
Earlier QLAAC comparison runs produced trials with zero successful responses and thousands of timeouts or all-503 trials	Superseded by the final five-trial tuned runs; the preliminary controller tripped OVERLOAD prematurely on transient worker saturation alone, causing runaway 503 shedding, whereas the tuned controller in src/adaptive_controller.c uses multi-factor thresholds and hysteresis.	experiment_qlaac_comparison.csv (preliminary, excluded)
Python tests skip silently when no server is listening	Found during report preparation: 11 of 23 tests skipped with no server running; all 23 passed against a live server. Recommendation: run them with the server started, or make the skip visible in the test report	Appendix B.2
QLAAC "shed" counter overstates 503s	Open issue: adaptive_controller_evaluate increments total_shed_requests whenever it is called in OVERLOAD, including by worker loops that ignore the decision; in a 5 s check qlaac_shed_requests was 29,783 while 503 responses sent were 26,978. Use total_overload_503_sent for shed counts	src/adaptive_controller.c, src/thread_pool.c; Appendix B.6

Testing. The suite contains 26 C unit tests (QLAAC 7, parser 9, queue 3, responses 2, metrics 5) and 23 Python tests (benchmark accounting 11, keep-alive 3, malformed requests 5, security 4). They verify specific behaviours, including controller transitions and hysteresis, framing errors, queue capacity, metrics accounting and path-traversal refusal. They are **functional tests, not performance measurements**, and they do not establish full RFC compliance or production security. Their execution output is in Appendix B.2.

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

Table 5.1: Metrics and their definitions

Metric	Definition (from the harness)	Why chosen
Successful throughput (req/s)	success_2xx divided by the 10 s duration; 503s, other errors and timeouts excluded	Measures useful work; prevents rejected requests from inflating throughput
Latency p50, p95, p99 (ms)	Per-trial percentile of client-observed latency of **2xx responses only**, index int(p·(n−1)) of the sorted samples; from just before sending the request to the last body byte	Median shows typical cost; p95/p99 show the tail that overload control targets
Shedding rate (%)	http_5xx / (success_2xx + http_5xx) per trial, as in verify_benchmark_statistics.py	Cost of protection: share of completed responses that were rejections
Share of attempted requests answered 2xx	success_2xx / attempted_requests	Separates attempts from successes; attempts include immediate retries
Timeouts, transport errors, invalid responses	Counts per trial; all zero except 18–38 timeouts per trial for QLAAC at C = 64	Detect failures other than deliberate 503s
Queue occupancy, worker utilisation, shed counts	Available server-side (/metrics) and used by the controller	Not recorded as time series in the raw data; discussed qualitatively only

Which requests enter each statistic. Latency percentiles describe only requests that returned 2xx. Under QLAAC at C = 64, about 72% of completed responses are 503s, whose latency is not included; the percentiles therefore describe the requests that were admitted (a form of survivorship), and must be read together with the shedding rate. Attempted requests are not an offered-load measure: the closed-loop clients reconnect immediately after a rejection, so attempts are inflated when the server rejects quickly.

5.2 Scenarios and parameter values

Table 5.2: Scenarios used in this report (all from the two "tuned" CSV files; 5 trials × 10 s each)

Scenario	Clients C	Relation to the 4 workers	Observed QLAAC behaviour (see 5.4)
S1 (low load)	2	C ≤ workers; no waiting	No shedding; throughput and latency indistinguishable from baseline
S2a (medium load)	8	C = 2 × workers; 4 connections wait	No 503s; throughput reduced
S2b (medium-high load)	32	C = 8 × workers; 28 connections wait	No 503s; throughput reduced
S3 (high load)	64	C = 16 × workers; 60 connections wait (queue 94% full)	About 72% of completed responses are 503

The faculty template provides one row for medium load; two medium points (C = 8 and C = 32) are reported because they behave differently from each other under the baseline (p99 of 29 ms against 204 ms). Datasets that were **not** used in Section 5.3, with the reason, are listed in

Appendix B.5: the legacy-schema files, the two-row smoke-test file and the preliminary two-trial comparison files. The package README quotes a different set of numbers (including a C = 1 tier) that do not appear in any CSV; the discrepancy is itemised in Appendix B.4. All values below were recomputed from the raw CSV files and agree with benchmark/verify_benchmark_statistics.py.

5.3 Results: tables and graphs, baseline vs proposed

Table 5.3 follows the faculty template. Means and sample standard deviations (n − 1) are across the five trials. "Improvement (%)" is positive when QLAAC is better: (QLAAC − baseline)/baseline for throughput and (baseline − QLAAC)/baseline for latency. Negative values mean QLAAC is worse. Units: throughput in successful 2xx requests per second; latency in milliseconds, for 2xx responses only; shedding rate in percent of completed responses.

Table 5.3: Baseline versus proposed (QLAAC), mean ± sample SD over 5 trials of 10 s

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
S1 (low load, C = 2)	Successful RPS (req/s)	13,929.24 ± 334.23	14,665.40 ± 610.30	+5.3
	p50 latency (ms)	0.13 ± 0.01	0.12 ± 0.00	+6.3
	p99 latency (ms)	0.31 ± 0.02	0.33 ± 0.05	-5.2
	Shedding rate (%)	0.00 ± 0.00	0.00 ± 0.00	n/a (baseline 0)
S2a (medium, C = 8)	Successful RPS (req/s)	6,629.38 ± 148.21	2,129.26 ± 50.35	-67.9
	p50 latency (ms)	0.54 ± 0.01	1.55 ± 0.02	-186.3
	p99 latency (ms)	29.09 ± 0.50	3.55 ± 0.06	+87.8
	Shedding rate (%)	0.00 ± 0.00	0.00 ± 0.00	n/a (baseline 0)
S2b (medium-high, C = 32)	Successful RPS (req/s)	6,860.38 ± 107.11	1,970.40 ± 55.83	-71.3
	p50 latency (ms)	0.52 ± 0.01	10.46 ± 0.26	-1896.9
	p99 latency (ms)	204.00 ± 2.94	16.55 ± 1.00	+91.9
	Shedding rate (%)	0.00 ± 0.00	0.00 ± 0.00	n/a (baseline 0)
S3 (high, C = 64)	Successful RPS (req/s)	6,828.52 ± 157.91	638.12 ± 126.44	-90.7
	p50 latency (ms)	0.52 ± 0.01	19.61 ± 6.35	-3642.4
	p99 latency (ms)	440.33 ± 9.21	161.18 ± 159.88	+63.4
	Shedding rate (%)	0.00 ± 0.00	72.14 ± 5.15	n/a (baseline 0)

Welch two-sample t-tests (n = 5 per group, indicative only) give $p = 0.054$ for throughput at C = 2 and $p < 10^{-7}$ for throughput at C = 8, 32 and 64; for p99, $p = 0.52$ at C = 2, $p < 10^{-7}$ at C = 8 and C = 32, and $p = 0.017$ at C = 64, where the QLAAC variance is very large. p95 values, average latency and the request-outcome counts are in Appendix B.4.

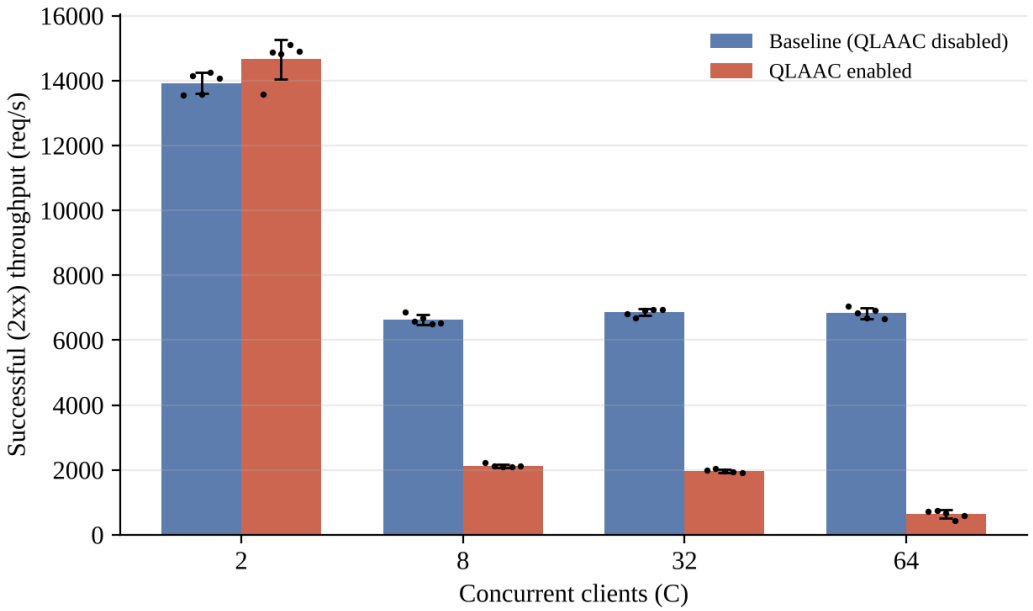


Figure 5.1: Successful (2xx) throughput, mean ± SD over 5 trials; dots are individual trials.

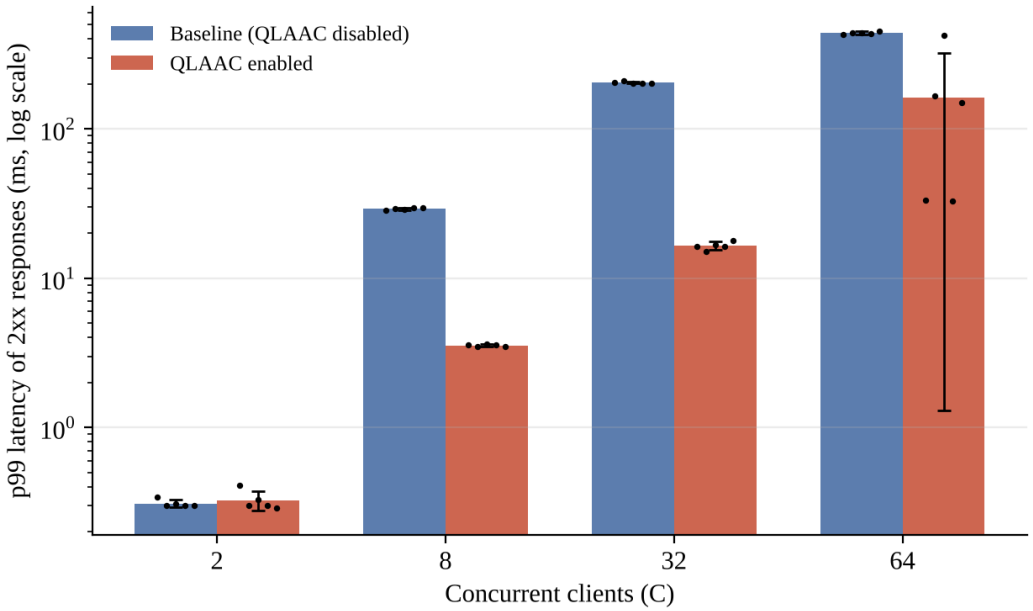


Figure 5.2: p99 latency of 2xx responses (log scale), mean ± SD; dots are individual trials. For QLAAC at C = 64 the trials fall into two groups (about 33 ms and 149–424 ms).

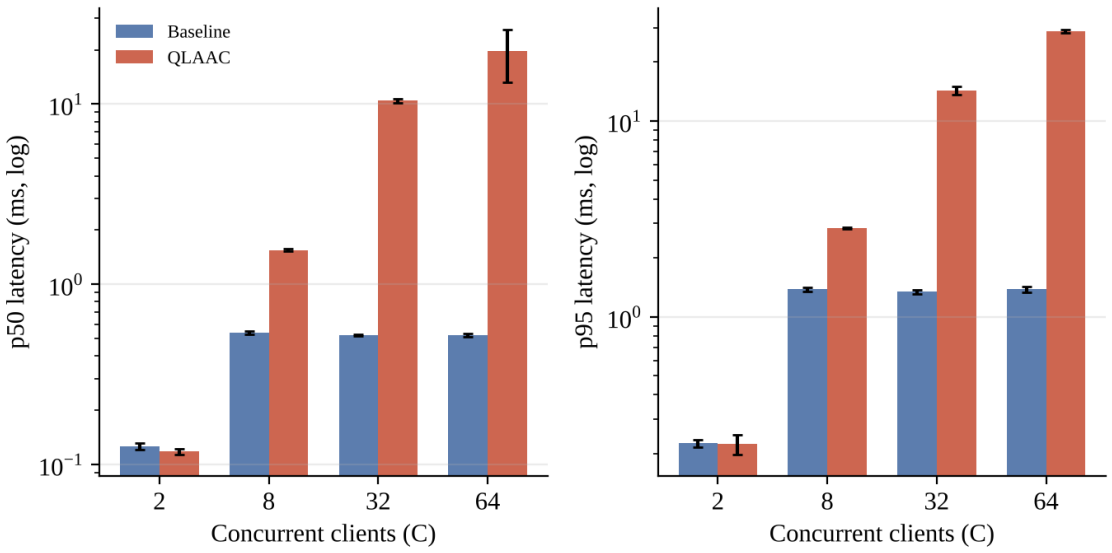


Figure 5.3: p50 and p95 latency of 2xx responses (log scale). QLAAC raises the median and p95 from C = 8 upward.

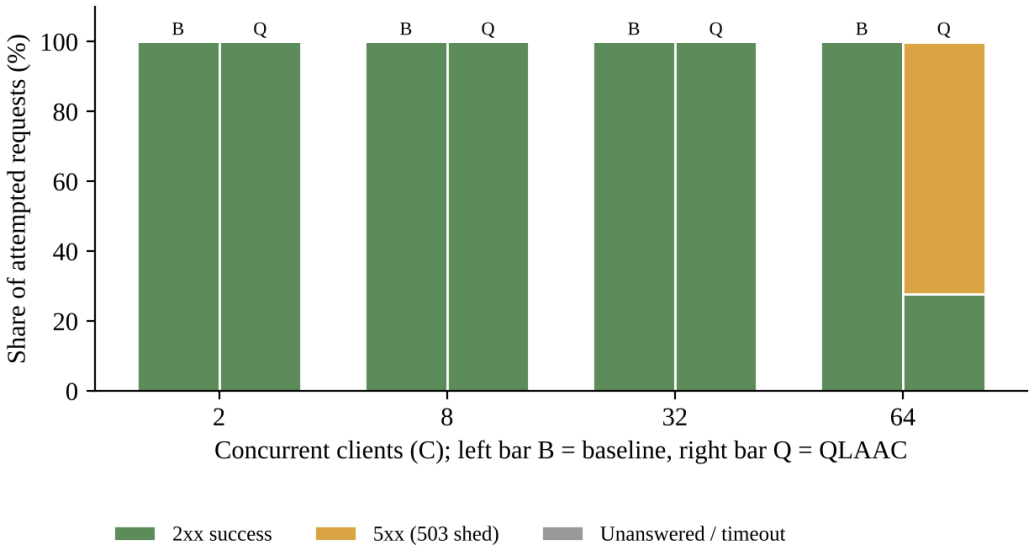


Figure 5.4: Outcome of attempted requests (means). At C = 64 under QLAAC only about 28% of attempted requests received a 2xx response; about 72% received a 503.

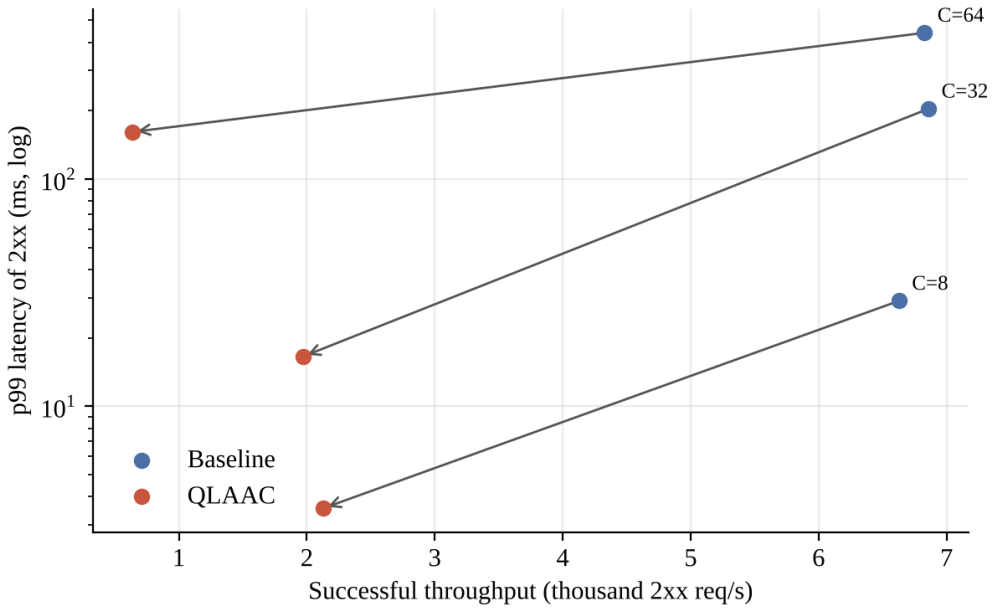


Figure 5.5: Throughput versus p99 trade-off. Arrows run from baseline (blue) to QLAAC (red) for C = 8, 32 and 64: tail latency falls and successful throughput falls with it.

5.4 Analysis and discussion

Low load (C = 2). With no more clients than workers, no connection ever waits. QLAAC stays in NORMAL, and differences in throughput (+5.3%) and p99 (−5.2%) are within trial-to-trial variation ($p = 0.054$ and 0.52). The controller adds no measurable cost here.

Why the baseline tail grows. In the baseline, every client holds a connection for up to 50 requests and a worker is pinned to a connection until it ends. With $C > 4$, the remaining $C - 4$ connections wait in the queue. The first request on a waiting connection therefore absorbs the queueing delay, while the other 49 requests on that connection are fast. Roughly 1 request in 50 (2%) is affected, which explains why the baseline p50 and p95 stay near 0.5 ms and 1.4 ms at every $C \geq 8$ while p99 climbs. A simple model reproduces the p99: the waiting time is about $\lceil (C - W)/W \rceil$ service rounds, each lasting $W \times 50 / X$ seconds ($W = 4$ workers, 50 requests per connection, X the measured throughput).

Table 5.4: Queue-wait model for the baseline p99 (the model uses the measured throughput, so it is a consistency check, not an independent prediction)

C	Throughput X (req/s)	Rounds ⌈(C−4)/4⌉	Predicted wait = rounds × 200 / X (ms)	Observed baseline p99 (ms)
8	6,629.38	1	30.2	29.09
32	6,860.38	7	204.1	204.00
64	6,828.52	15	439.3	440.33

The agreement is close, which supports the interpretation that baseline tail latency is connection-queue wait under pinned keep-alive connections. As a further check, Little's law for a closed system predicts a mean latency near C/X; for the baseline at C = 64 this gives 9.4 ms against an observed mean of about 9.4 ms (trial values 9.09–9.62 ms). The load generator and server share one host, so these absolute values include client-side overheads.

What QLAAC does at C = 8 and C = 32. The controller entered CAUTION with no 503s (zero 5xx in all ten trials). The source and a short supplementary check (Appendix B.6) show why: with more than four persistent connections all four workers are always busy, so worker utilisation is 1.0, and the controller treats worker saturation alone as a CAUTION condition. In CAUTION each connection is closed after one request, so every request needs a new TCP connection, an accept, a queue hop and a close. The tail shrinks because connections rotate through the workers instead of waiting for 50-request runs: p99 falls by 87.8% (C = 8) and 91.9% (C = 32). The price is paid everywhere else: successful throughput falls by 67.9% and 71.3%, p50 rises from 0.54 to 1.55 ms and from 0.52 to 10.46 ms, and p95 rises from 1.34 to 14.31 ms at C = 32. In this workload the controller converts a bimodal latency distribution (most requests very fast, a few very slow) into a uniformly moderate one at much lower throughput. Note also that the client's latency timer starts after the TCP connection is established, so the connection set-up cost that reduces throughput is not visible in the latency columns.

What QLAAC does at C = 64. The queue holds about 60 connections (94% of capacity), which exceeds the 0.90 critical threshold, so the controller enters OVERLOAD and the listener answers new connections with 503. Clients reconnect immediately, so rejected attempts are replaced by new attempts. Over the five trials, 71.998% of attempted requests and 72.14 ± 5.15% of completed responses were 5xx; only 27.8% of attempted requests received a 2xx response. Successful throughput fell by 90.7% (6,828.52 to 638.12 req/s). Admitted requests had a p50 of 19.61 ms (against 0.52 ms) and a p95 of 28.76 ms (against 1.38 ms). The p99 of admitted requests fell from 440.33 ms to a mean of 161.18 ms, but with a standard deviation of 159.88 ms: two trials were near 33 ms and three were between 149 and 424 ms (Figure 5.2). A mean improvement of 63.4% is therefore not a reliable summary; the median trial improved by about 66% (149.39 ms against 438.40 ms), but the distribution is clearly unstable. The slow recovery rule (five healthy windows) means rejection continues after the queue has drained. A short supplementary run (Appendix B.6) logged NORMAL to CAUTION at 0 s, CAUTION to OVERLOAD at 5 s and recovery to CAUTION at 10 s, consistent with alternating shed and

admit periods, but the final experiment did not record controller state over time, so this mechanism is an inference.

Trade-off summary. QLAAC reduced p99 in every loaded scenario, but in every loaded scenario it also reduced successful throughput by more than two thirds, raised the median and p95, and at C = 64 rejected most arriving work. Whether this is desirable depends on the objective: if a tail-latency target for admitted requests is paramount and clients can retry later, it protects the tail; if goodput or median latency matters, the baseline is far better in this testbed.

5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails

Strengths. The controller is small (211 lines), uses only in-process signals, is unit-tested for transitions and hysteresis, produces a large and statistically clear reduction in p99 at C = 8 and C = 32, and fails safe: it adds no measurable cost at low load. The benchmark accounting is strict about what counts as success, which makes the cost of protection visible rather than hidden.

Weaknesses and verified limitations.

- **Over-eager CAUTION.** Worker saturation alone raises CAUTION. With persistent connections and C > W this is always true, so QLAAC throttles keep-alive even when the queue is short and the server is making steady progress (C = 8: throughput −67.9% with no queue overflow risk).
- **Latency signal blind to queue wait.** The server-side latency starts when request bytes are available, after a connection has left the queue. In the supplementary check the p95 seen by the controller was about 0.05–0.12 ms at the moment of each transition, far below the 50 ms and 150 ms thresholds. Queue occupancy and worker utilisation, not latency, triggered every observed transition, so the "latency-aware" part was inactive in this workload.
- **Unstable tail at overload.** p99 at C = 64 has a coefficient of variation of about 99% across trials; five trials cannot characterise this.
- **Rejections are not retried sensibly.** Clients reconnect immediately, which creates a retry storm against the listener; Retry-After is not honoured by the harness.

- **Coarse control.** One-second sampling and five-window recovery are slow compared with request times of about 0.1 ms; thresholds were not tuned and no sensitivity study was done.
- **Counter inconsistency.** qlaac_shed_requests overcounts relative to 503 responses sent (Section 4.4).

Measurement limitations. One workload (a tiny JSON health endpoint, no disk I/O or large bodies); one host with client and server competing for the same CPUs; a Python threaded load generator whose own overhead and interpreter lock affect absolute throughput (for example, baseline throughput at C = 8 is below its value at C = 2); closed-loop clients, so offered load adapts to server speed; five trials per point; no open-loop or burst arrival experiment; no recovery-time measurement; client latency excludes TCP connection set-up; hardware and exact launch commands not recorded. Absolute numbers should be read as relative comparisons within this testbed.

Conditions under which QLAAC fails to improve performance or reduces successful throughput. (i) When concurrency does not exceed the worker count (no difference, C = 2). (ii) Whenever the objective is throughput, median or p95 latency rather than p99: QLAAC lowered throughput by 68–91% and raised p50 at $C \geq 8$. (iii) When most load comes from clients that retry immediately, since shedding then multiplies connection attempts. (iv) When the tail is caused by queue wait that the controller's own latency signal does not see. (v) When a stable p99 for admitted requests is required at overload: at C = 64 the p99 was unstable.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings

A multi-threaded HTTP/1.1 server was implemented in C17 on raw TCP sockets, comprising a listener, bounded connection queue, four-worker pool, streaming parser for a documented subset of RFC 9112, router, static-file delivery with traversal checks, metrics and a dashboard, together with 49 automated tests that pass when a server instance is running. QLAAC, a three-state admission controller with hysteresis, was added and compared against the same server with the controller disabled, using a harness that separates 2xx responses from every other outcome. The quantitative findings (five 10-second trials per point, mean ± sample SD) are:

- C = 2: no significant difference (throughput 13,929 ± 334 vs 14,665 ± 610 req/s; p99 0.31 vs 0.33 ms).
- C = 8: p99 29.09 → 3.55 ms (−87.8%), successful throughput 6,629 → 2,129 req/s (−67.9%), no 503s.
- C = 32: p99 204.00 → 16.55 ms (−91.9%), throughput 6,860 → 1,970 req/s (−71.3%), no 503s; p50 0.52 → 10.46 ms.
- C = 64: p99 440.33 → 161.18 ± 159.88 ms (−63.4%, unstable), throughput 6,829 → 638 req/s (−90.7%), 72.14 ± 5.15% of completed responses were 503.
- Baseline tail latency is explained by connection-queue wait behind pinned keep-alive connections (model agreement in Table 5.4).

QLAAC therefore trades successful throughput, median latency and a large share of requests for lower tail latency of admitted requests. It should not be described as universally better.

6.2 Limitations

Implementation: HTTP/1.1 support is a documented subset (no chunked bodies, TLS or range requests); blocking worker threads limit connection scalability; the shed counter overcounts; two configuration entries are inconsistent with the code. Measurement: single host and workload, Python load generator, closed-loop clients, five trials per point, no recorded hardware details, no time-series of controller state. Scalability beyond 64 clients, other workloads (large files, POST bodies), other thread/queue sizes and production readiness were not evaluated, and no security audit beyond the listed tests was performed.

6.3 Future enhancements

The following are proposals, not completed work:

- Measure latency from accept time (or record queue wait per connection) so that the latency signal sees queueing delay.
- Make worker saturation alone insufficient for keep-alive throttling, or scale the throttle with queue depth; study threshold sensitivity systematically.
- Add an open-loop load generator (for example wrk, or a fixed-rate client) with Retry-After-aware clients, and record controller state, queue depth and shed counts as time series, including recovery time.
- Evaluate other workloads (static files of several sizes, POST echo) and other worker/queue configurations, with more trials and confidence intervals.
- Fix the shed counter, make skipped tests visible, and extend parser coverage (chunked requests, range requests); consider an epoll-based design for comparison.

7. REFERENCES

References (IEEE style)

[1] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP/1.1," Internet Engineering Task Force, RFC 9112 (STD 99), Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9112.html

[2] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP Semantics," Internet Engineering Task Force, RFC 9110 (STD 97), Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html

[3] W. Eddy, Ed., "Transmission Control Protocol (TCP)," Internet Engineering Task Force, RFC 9293 (STD 7), Aug. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293.html

[4] M. Welsh, D. Culler, and E. Brewer, "SEDA: An architecture for well-conditioned, scalable Internet services," in Proc. 18th ACM Symp. Operating Systems Principles (SOSP), Banff, AB, Canada, 2001, pp. 230–243. doi: 10.1145/502034.502057.

[5] M. Welsh and D. Culler, "Adaptive overload control for busy Internet servers," in Proc. 4th USENIX Symp. Internet Technologies and Systems (USITS), Seattle, WA, USA, Mar. 2003, pp. 45–60. [Online]. Available: https://www.usenix.org/conference/usits-03/adaptive-overload-control-busy-internet-servers

[6] B. Schroeder and M. Harchol-Balter, "Web servers under overload: How scheduling can help," ACM Trans. Internet Technol., vol. 6, no. 1, pp. 20–52, Feb. 2006. doi: 10.1145/1125274.1125276.

[7] D. Pariag, T. Brecht, K. Harji, P. Buhr, A. Shukla, and D. Cheriton, "Comparing the performance of web server architectures," in Proc. 2nd ACM SIGOPS/EuroSys European Conf. Computer Systems (EuroSys), Lisbon, Portugal, 2007, pp. 231–243. doi: 10.1145/1272996.1273019.

[8] K. Nichols and V. Jacobson, "Controlling queue delay," ACM Queue, vol. 10, no. 5, pp. 20–34, May 2012. doi: 10.1145/2208917.2209336.

[9] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," ACM Queue, vol. 9, no. 11, pp. 40–54, Nov. 2011. doi: 10.1145/2063166.2071893.

[10] J. Dean and L. A. Barroso, "The tail at scale," Commun. ACM, vol. 56, no. 2, pp. 74–80, Feb. 2013. doi: 10.1145/2408776.2408794.

[11] J. D. C. Little, "A proof for the queuing formula: L = λW," Oper. Res., vol. 9, no. 3, pp. 383–387, Jun. 1961. doi: 10.1287/opre.9.3.383.

APPENDIX A: IMPORTANT SOURCE CODE OR CONFIGURATION LISTINGS

Excerpts are shortened for the report; line comments are condensed. The full code is in the supplied repository.

A.1 QLAAC overload and caution conditions (src/adaptive_controller.c)

This is the core decision logic. It shows that worker saturation alone is a caution condition and that overload requires queue, latency, saturation-plus-queue/latency or sustained pressure.

```
bool sustained_pressure_candidate = (worker_saturated && queue_occupancy_ratio >= 0.50) ||
                                    (worker_saturated && latency_elevated) ||
                                    (queue_growing && latency_elevated);
...
bool is_overload_condition = queue_critical || latency_critical ||
                             (worker_saturated && queue_growing) ||
                             (worker_saturated && latency_elevated) ||
                             sustained_pressure_overload;
bool is_caution_condition = is_overload_condition || queue_growing ||
                            latency_elevated || worker_caution || worker_saturated;
```

A.2 Accept-time admission and 503 shedding (src/server.c)

The listener asks the controller for a decision for every accepted connection; in OVERLOAD it answers 503 itself, and otherwise attempts a non-blocking push to the bounded queue (a full queue also yields a 503).

```
QlaacDecision decision = adaptive_controller_evaluate(&ctx->controller);
if (decision == QLAAC_ACTION_REJECT_OVERLOAD) {
    metrics_record_connection_rejected(&ctx->metrics);
    const char *msg_503 = "HTTP/1.1 503 Service Unavailable\r\n"
        "Content-Type: text/html; charset=utf-8\r\nContent-Length: 96\r\n"
        "Connection: close\r\nRetry-After: 5\r\n\r\n<html>...</html>\n";
    ... tcp_send_all(client.fd, msg_503, strlen(msg_503), deadline, &sent_bytes);
    tcp_lingering_close(&client.fd, 50);
    continue;
}
int push_rc = connection_queue_push(&ctx->queue, &client, 0);   /* non-blocking */
if (push_rc != 0) { /* queue full: 503 with Retry-After: 2 */ }
```

A.3 CAUTION keep-alive throttle in the worker loop (src/thread_pool.c)

This explains the throughput drop at C = 8 and C = 32: once the state is CAUTION, each connection is closed after one more request. In OVERLOAD the same call only increments the shed counter and the decision is ignored by the worker.

```
QlaacDecision qdec = adaptive_controller_evaluate(&ctx->controller);
if (qdec == QLAAC_ACTION_THROTTLE_KEEPALIVE) {
    keepalive_timeout_sec = 1;
    max_requests = 1;
}
```

A.4 Configuration (config/server.conf.example, QLAAC section)

Threshold values used in Table 3.1; adaptive_control_enabled is false in the example file, which is why the baseline is the default.

```
worker_threads = 4
connection_queue_capacity = 64
keepalive_max_requests = 100
keepalive_timeout_sec = 5
adaptive_control_enabled = false
sampling_window_ms = 1000
queue_caution_threshold = 0.70
queue_overload_threshold = 0.90
latency_caution_ms = 50.0
latency_overload_ms = 150.0
worker_caution_threshold = 0.85
worker_overload_threshold = 1.00
sustained_pressure_threshold = 3
recovery_sample_count = 5
```

A.5 Harness accounting (benchmark/run_experiment.py)

The harness counts a response as a success only when the parsed status code is 2xx, and computes throughput and latency from those responses only.

```
if 200 <= status_code < 300:
    local_2xx += 1
    local_latencies.append(latency_ms)
elif 300 <= status_code < 400:  local_3xx += 1
elif 400 <= status_code < 500:  local_4xx += 1
elif 500 <= status_code < 600:  local_5xx += 1
...
successful_rps = success_2xx / duration_sec
p99 = success_latencies[int(0.99 * (count - 1))]      # N/A if no 2xx
```

A.6 Commands

Build and test (Makefile targets as supplied; start the server first so that the Python tests do not skip):

```
make                      # builds bin/server
make test                 # builds and runs the 5 C unit-test binaries, then the Python tests
./bin/server -c config/server.conf.example -p 8080          # baseline
./bin/server -c config/server.conf.example -p 8080 -a       # QLAAC enabled
python3 -m unittest discover -s tests -p "test_*.py" -v     # with a server running
python3 benchmark/verify_benchmark_statistics.py            # recompute Chapter 5 statistics
```

APPENDIX B: ADDITIONAL SCREENSHOTS, PACKET CAPTURES, RAW RESULT TABLES

B.1 Application screenshots (captured from the running server)

These captures were taken from the live dashboard (supplied files 02, 03, 04 and 05). They show functional behaviour, not benchmark results. The capture in Figure B.4 shows the controller panel in the NORMAL state at idle, not under overload.

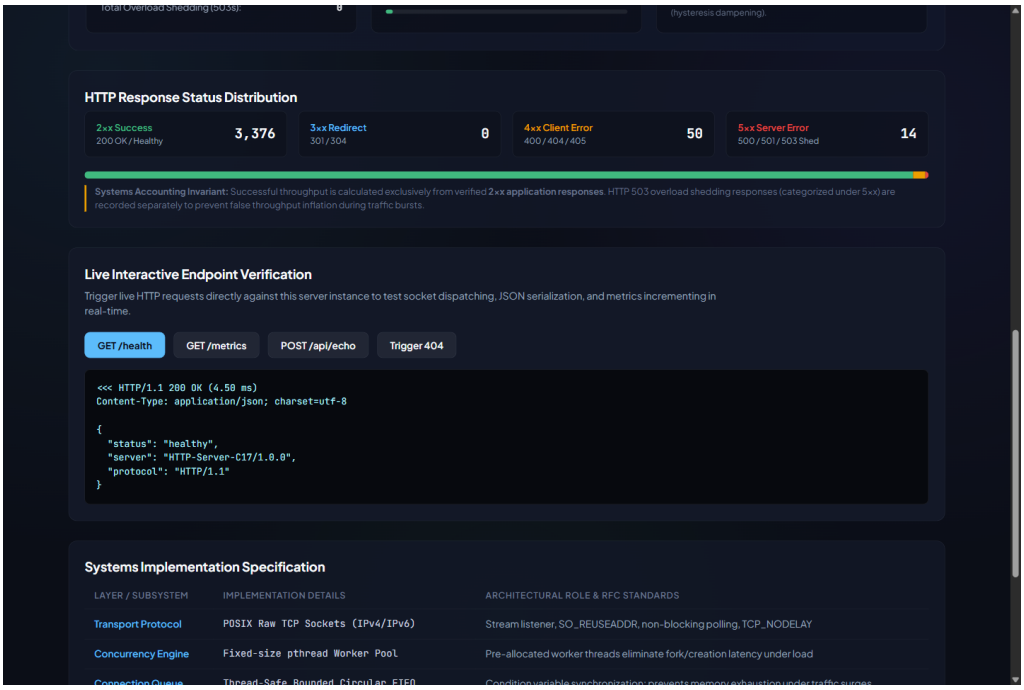


Figure B.1: API console: GET /health returning HTTP/1.1 200 OK with the JSON health payload (4.50 ms round trip as displayed), and the status-class distribution (2xx 3,376; 4xx 50; 5xx 14).

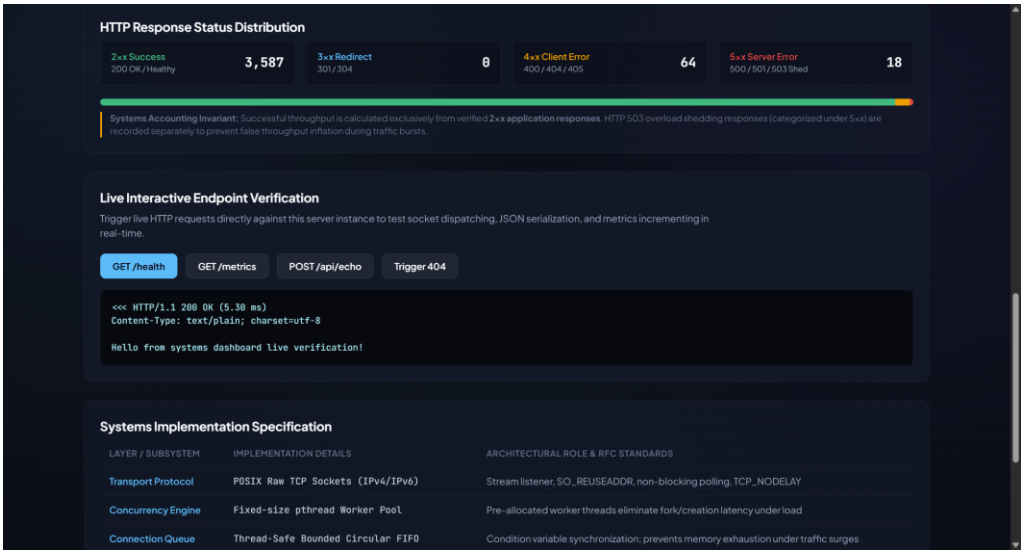


Figure B.2: API console: POST /api/echo returning HTTP/1.1 200 OK with the request body echoed (5.30 ms as displayed).

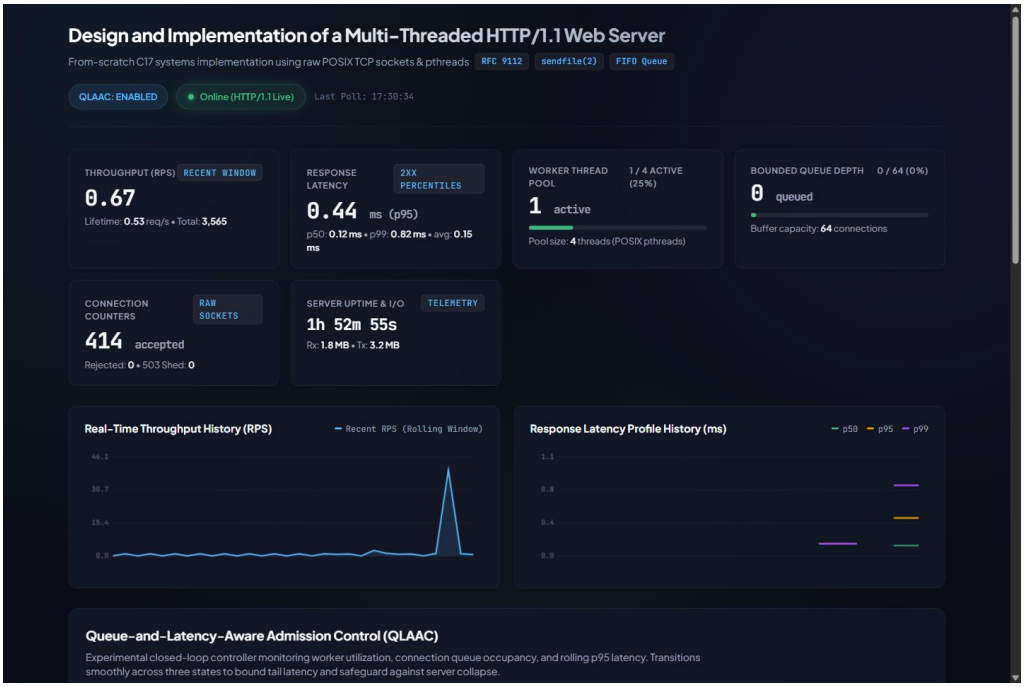


Figure B.3: Dashboard after a short burst of requests: throughput spike, 2xx percentiles p50 0.12 ms, p95 0.44 ms, p99 0.82 ms (local manual burst; not part of the benchmark).

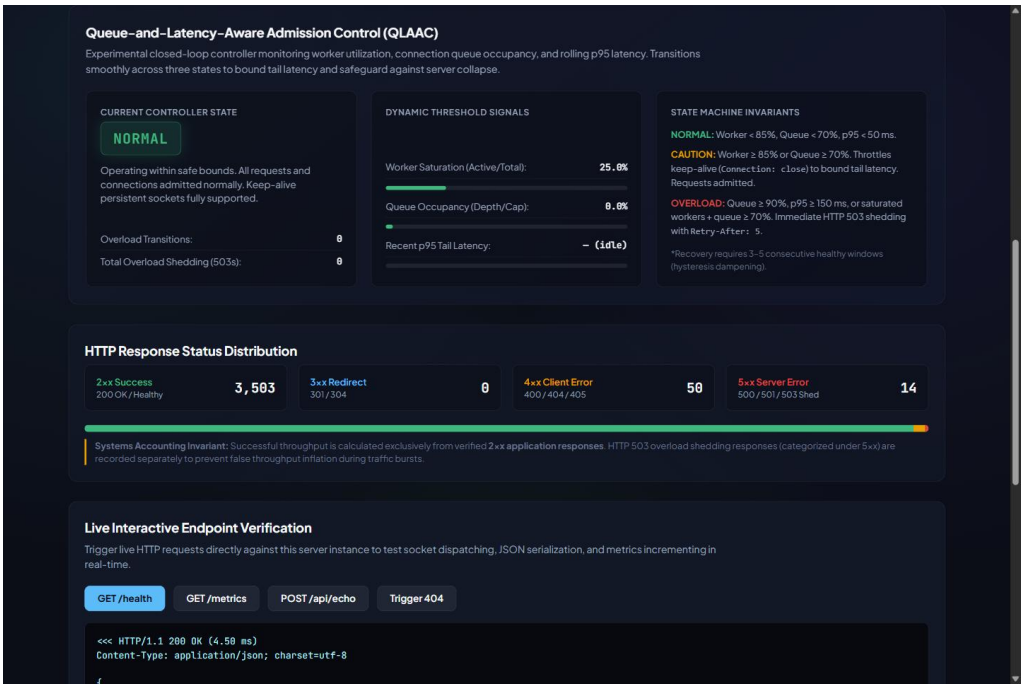


Figure B.4: QLAAC controller panel (state NORMAL, worker saturation 25%, queue 0%, zero shed) with the displayed state-machine invariants.

B.2 Test execution output

The supplied image 06_test_results.png is **not used**: according to the available execution history it was produced by an image-generation script, not captured from a terminal, so it cannot serve as evidence of test execution. The output below was produced by re-running the tests on the Linux verification environment of Table 4.1 on 17 October 2026 (GCC 13.3.0,

Python 3.13.16). It is the textual output of make test (C unit tests) and of the Python tests run with a server listening on port 8080; compiler command lines and QLAAC log lines are omitted.

```
$ make test
Executing bin/test_controller...   [PASS] x7   (--- All QLAAC Controller Tests Passed ---)
Executing bin/test_parser...       [PASS] x9   (--- All HTTP Parser Tests Passed ---)
Executing bin/test_queue...        [PASS] x3   (--- All Connection Queue Tests Passed ---)
Executing bin/test_responses...    [PASS] x2   (--- All HTTP Response Tests Passed ---)
Executing bin/test_server_metrics... [PASS] x5 (--- All Server Metrics Tests Passed ---)
All C unit test skeletons executed successfully.            -> 26 / 26 passed


$ ./bin/server -c config/server.conf.example -p 8080 &
$ python3 -m unittest discover -s tests -p "test_*.py" -v
Ran 23 tests in 0.108s
OK                                                          -> 23 / 23 passed, 0 skipped


(same command with NO server listening:  Ran 23 tests ... OK (skipped=11))
```

Total: 49 of 49 tests pass when a server is running. The individual test names are those listed in results/test_evidence/test_run_summary.txt, which agrees with this re-run.

B.3 Packet captures

No packet captures (pcap files or Wireshark screenshots) were included in the supplied materials, and none are presented in this report.

B.4 Raw result tables

Per-trial values from the two CSV files used in Chapter 5 (attempted requests, 2xx, 5xx, timeouts, successful throughput and latency percentiles of 2xx responses). All other columns (3xx, 4xx, transport errors, invalid responses) are zero in every trial.

Table B.1: Raw per-trial data, baseline (experiment_baseline_tuned_comparison.csv); latencies in ms

C	Trial	Attempted	2xx	5xx	Timeouts	RPS (2xx)	p50	p95	p99
2	1	135,667	135,667	0	0	13566.70	0.13	0.24	0.34
2	2	141,566	141,566	0	0	14156.60	0.12	0.22	0.30
2	3	135,737	135,737	0	0	13573.70	0.13	0.23	0.31
2	4	142,665	142,665	0	0	14266.50	0.12	0.22	0.30
2	5	140,827	140,827	0	0	14082.70	0.13	0.22	0.30
8	1	68,686	68,686	0	0	6868.60	0.52	1.32	28.35
8	2	65,718	65,718	0	0	6571.80	0.54	1.40	29.13
8	3	66,729	66,729	0	0	6672.90	0.54	1.37	28.91
8	4	65,104	65,104	0	0	6510.40	0.55	1.41	29.48
8	5	65,232	65,232	0	0	6523.20	0.55	1.41	29.60
32	1	68,193	68,193	0	0	6819.30	0.53	1.33	204.59
32	2	66,908	66,908	0	0	6690.80	0.53	1.40	208.90
32	3	69,097	69,097	0	0	6909.70	0.52	1.32	202.04
32	4	69,404	69,404	0	0	6940.40	0.52	1.34	202.40
32	5	69,417	69,417	0	0	6941.70	0.52	1.32	202.07
64	1	70,437	70,437	0	0	7043.70	0.51	1.31	429.52
64	2	68,354	68,354	0	0	6835.40	0.52	1.38	438.40
64	3	66,915	66,915	0	0	6691.50	0.53	1.42	442.85

C	Trial	Attempted	2xx	5xx	Timeouts	RPS (2xx)	p50	p95	p99
64	4	69,105	69,105	0	0	6910.50	0.52	1.37	436.49
64	5	66,615	66,615	0	0	6661.50	0.54	1.43	454.40

Table B.2: Raw per-trial data, QLAAC (experiment_qlaac_tuned_comparison.csv); latencies in ms

C	Trial	Attempted	2xx	5xx	Timeouts	RPS (2xx)	p50	p95	p99
2	1	135,919	135,919	0	0	13591.90	0.12	0.27	0.41
2	2	148,823	148,823	0	0	14882.30	0.12	0.21	0.30
2	3	148,222	148,222	0	0	14822.20	0.11	0.22	0.33
2	4	151,177	151,177	0	0	15117.70	0.12	0.21	0.30
2	5	149,129	149,129	0	0	14912.90	0.12	0.21	0.29
8	1	22,177	22,177	0	0	2217.70	1.51	2.81	3.59
8	2	21,164	21,164	0	0	2116.40	1.56	2.81	3.47
8	3	20,913	20,913	0	0	2091.30	1.57	2.85	3.61
8	4	21,089	21,089	0	0	2108.90	1.54	2.87	3.60
8	5	21,120	21,120	0	0	2112.00	1.55	2.82	3.50
32	1	19,901	19,901	0	0	1990.10	10.35	14.11	16.37
32	2	20,548	20,548	0	0	2054.80	10.09	13.35	15.17
32	3	19,609	19,609	0	0	1960.90	10.49	14.50	16.81
32	4	19,371	19,371	0	0	1937.10	10.62	14.43	16.42
32	5	19,091	19,091	0	0	1909.10	10.77	15.17	17.96
64	1	23,232	7,318	15,876	38	731.80	19.97	28.79	33.18
64	2	23,336	7,542	15,776	18	754.20	22.39	28.08	166.30
64	3	22,376	6,683	15,674	19	668.30	23.99	29.33	424.26
64	4	22,134	4,407	17,706	21	440.70	23.13	29.34	32.77
64	5	23,338	5,956	17,345	37	595.60	8.57	28.26	149.39

Table B.3: Additional aggregates (means over 5 trials; ± is sample SD)

C	Config	p95 (ms)	Avg latency (ms)	Attempted	2xx	5xx	Timeouts	2xx / attempted (%)
2	Baseline	0.23 ± 0.01	0.14 ± 0.00	139,292	139,292	0	0.0	100.0
2	QLAAC	0.22 ± 0.03	0.13 ± 0.00	146,654	146,654	0	0.0	100.0
8	Baseline	1.38 ± 0.04	1.18 ± 0.02	66,294	66,294	0	0.0	100.0
8	QLAAC	2.83 ± 0.03	1.63 ± 0.02	21,293	21,293	0	0.0	100.0
32	Baseline	1.34 ± 0.03	4.64 ± 0.07	68,604	68,604	0	0.0	100.0
32	QLAAC	14.31 ± 0.66	10.48 ± 0.30	19,704	19,704	0	0.0	100.0
64	Baseline	1.38 ± 0.05	9.39 ± 0.22	68,285	68,285	0	0.0	100.0
64	QLAAC	28.76 ± 0.59	24.55 ± 2.51	22,883	6,381	16,475	26.6	27.8

Discrepancy between the package README and the raw data. The supplied CLAUDE_PACKAGE_README.md quotes a table of "verified" values that does not match the CSV files or verify_benchmark_statistics.py. This report uses the CSV-derived values. The differences are:

Table B.4: Package README claims compared with values recomputed from the CSV files

Item	Package README	Raw CSV / source
Concurrency tiers	1, 8, 32, 64	2, 8, 32, 64 (no C = 1 in the tuned files)

Item	Package README	Raw CSV / source
Baseline throughput, C = 64	1,215.80 ± 45.10 req/s	6,828.52 ± 157.91 req/s
QLAAC throughput, C = 64	182.40 ± 16.50 req/s	638.12 ± 126.44 req/s
QLAAC shedding, C = 64	≈ 87.5%	72.14 ± 5.15%
QLAAC p99, C = 64	38.40 ± 3.20 ms (< 40 ms)	161.18 ± 159.88 ms
QLAAC throughput, C = 8 / 32	880.60 / 1,140.20 req/s (≈ baseline)	2,129.26 / 1,970.40 req/s (−67.9% / −71.3% vs baseline)
Baseline queue	"Unbounded"	Bounded, capacity 64 (src/connection_queue.c)
06_test_results.png	"Genuine terminal output"	Script-generated image; excluded (B.2)
Python tests	All 49 pass	Pass only when a server is listening; 11 of 23 skip otherwise

B.5 Datasets not used for the comparison

Table B.5: Raw CSV files in the package that are excluded from Chapter 5, and why

File	Content	Reason for exclusion
benchmark_results.csv	Legacy schema, 3 rows (C = 1, 4, 16)	Legacy accounting counted any "HTTP/1.1" response as success; no per-class columns
fixed_policy_8080.csv, fixed_policy_high_load.csv, qlaac_enabled_8081.csv, qlaac_enabled_high_load.csv	Legacy schema, 6–9 rows each	Same legacy accounting; not comparable with the corrected schema (docs/experiment-methodology.md section 7)
benchmark_results_corrected.csv	2 rows (C = 1 and 2, one 2 s trial each)	Smoke test of the corrected harness; no repetition
experiment_baseline_comparison.csv	Corrected schema, C = 2, 8, 32, 64, 2 trials of 3 s	Preliminary short run; superseded by the 5 × 10 s data
experiment_qlaac_comparison.csv	Corrected schema, 16 rows in two groups	Preliminary; several trials with zero successful responses and thousands of timeouts or all-503 results; behaviour of that earlier version not documented

B.6 Supplementary mechanism check (not used in Chapter 5)

To understand why QLAAC reduced throughput at C = 8 without shedding, the server was started with and without -a on the verification environment (2 logical CPUs, a different machine from the student's) and the same harness was run for a single 5 s trial at C = 8 and C = 64. These numbers are from different hardware and a single trial, and are used only to confirm the mechanism qualitatively.

Table B.6: Single 5 s trials on the verification environment (not comparable with Chapter 5)

C	Configuration	2xx	5xx	Successful RPS	p50 (ms)	p99 (ms)
8	Baseline	110,813	0	22,162.6	0.14	8.77
8	QLAAC	41,693	0	8,338.6	0.36	2.03
64	Baseline	115,651	0	23,130.2	0.14	127.67
64	QLAAC	4,135	26,953	827.0	2.65	155.09

The controller log from the QLAAC run read: "NORMAL -> CAUTION (queue=0.06, worker_util=1.00, p95=0.05 ms)", then "CAUTION -> OVERLOAD (queue=0.86, worker_util=1.00, p95=0.10 ms)" about 5 s later, then "OVERLOAD -> CAUTION (queue=0.00, worker_util=0.00, p95=0.12 ms)" after a further 5 s. At the end, /metrics reported qlaac_shed_requests = 29,783 against total_overload_503_sent = 26,978, which is the counter inconsistency noted in Section 4.4. The transitions were triggered by worker saturation and queue occupancy while the p95 seen by the controller stayed near 0.1 ms.